

AI Customer Support Triage Platform

Overview

This project is a two-tier support triage platform with:

- `frontend/` — Angular SPA
- `backend/` — ASP.NET Core Web API
- JWT-based authentication
- Role-based access control for customers, agents, and admins
- Support for ticket creation, comments, attachments, categories, teams, and AI-assisted reply suggestions

The frontend communicates with the backend at `http://localhost:5000/api`.

Backend Architecture

Core components

- `backend/Program.cs`
 - Configures Serilog logging
 - Registers controllers, JWT auth, authorization, CORS, Swagger/OpenAPI
 - Supports optional EF Core SQL Server and Hangfire background jobs
 - Seeds in-memory store if SQL Server is disabled
 - Starts the API on `http://localhost:5000`
- `backend/Controllers`
 - `AuthController.cs`
 - `TicketsController.cs`
 - `CommentsController.cs`
 - `AttachmentsController.cs`
 - `CategoriesController.cs`
 - `TeamsController.cs`
 - `UsersController.cs`
 - `DashboardController.cs`
- `backend/DTOs`
 - `AuthDtos.cs`
 - `TicketDtos.cs`
 - `TeamDtos.cs`
- `backend/Services`
 - `Store.cs` — in-memory domain store for users, teams, categories, tickets, comments, attachments, history
 - `Auth.cs` — user extraction, role checks, ticket access enforcement
 - `Api.cs` — standardized response formatting and DTO mapping
 - `TicketHistoryService.cs` — audit trail entries
 - `TicketJobDispatcher.cs` — background job enqueueing
 - `AIReplyService.cs` — AI-generated reply suggestion support

Data persistence model

- Default runtime store is in-memory dictionaries inside `backend/Services/Store.cs`.
 - Optional persistence with SQL Server when `Features:UseSqlServer` is enabled.
 - Optional background work via Hangfire when `Features:UseHangfire` is enabled.
-

Frontend Architecture

Core components

- `frontend/src/environments/environment.ts`
 - `apiUrl: 'http://localhost:5000/api'`
 - `frontend/src/app/app.config.ts`
 - Registers global HTTP interceptors
 - `frontend/src/app/interceptors/auth.interceptor.ts`
 - Adds `Authorization: Bearer <token>` header to API requests for valid JWTs
 - Frontend services
 - `AuthService` — register, login, session persistence
 - `TicketService` — ticket list, detail, create, attachments, comments, AI reply, status/category/priority/assignment updates
 - `AdminDataService` — teams, categories, agents lists and CRUD operations
 - `DashboardService` — dashboard stats and agent workload data
 - Models in `frontend/src/app/models`
 - `User`, `ticket`, `pagination`, `comment`, `attachment`, `history` interfaces
-

API Contracts

Authentication

POST `/api/auth/register`

- Request body:
 - `fullName: string`
 - `email: string`
 - `password: string`
- Response:
 - `token: string`
 - `user: { id, fullName, email, role }`

POST `/api/auth/login`

- Request body:
 - email: string
 - password: string
- Response: same as register

GET /api/auth/me

- Response: current UserDto

Ticket APIs

POST /api/tickets

- Request body:
 - title: string
 - description: string
 - categoryId?: number
- Response: created ticket summary

GET /api/tickets

- Query parameters:
 - page?: number
 - pageSize?: number
 - status?: string
 - priority?: string
 - search?: string
- Response: PaginatedResponse<TicketListItem>

GET /api/tickets/{id}

- Response: TicketDetailResponse

PUT /api/tickets/{id}/status

- Request body: { status: string }

PUT /api/tickets/{id}/priority

- Request body: { priority: string }

PUT /api/tickets/{id}/category

- Request body: { categoryId: number }

PUT /api/tickets/{id}/assign

- Request body: { agentId: string, teamId: number }

POST /api/tickets/{id}/ai-reply

- Response: { reply: string }

Comments

GET /api/tickets/{ticketId}/comments

POST /api/tickets/{ticketId}/comments

- Request body: { message: string, isInternal: boolean }

Attachments

POST /api/tickets/{ticketId}/attachments

- Multipart form upload with field file
- Limits: max 5 MB, up to 3 attachments per ticket

GET /api/attachments/{id}

Categories

GET /api/categories

POST /api/categories

- Request body: { name: string, defaultTeamId: number }

PUT /api/categories/{id}

- Request body: { name: string, defaultTeamId: number }

DELETE /api/categories/{id}

Teams

GET /api/teams

POST /api/teams

- Request body: { name: string, description?: string }

POST /api/teams/{id}/members

- Request body: { userId: string, role: string }

PUT /api/teams/{id}

- Request body: { name: string, description?: string }

DELETE /api/teams/{id}

Users

GET /api/users

- Optional query: role=agent

Dashboard

GET /api/dashboard/stats

GET /api/dashboard/agents

Domain Models

User

- Id: Guid
- FullName: string
- Email: string
- PasswordHash: string
- Role: string (Customer, Agent, Admin)
- IsActive, timestamps

Ticket

- Id: long
- Title, Description
- Status
- Priority
- CategoryId
- AssignedAgentId
- AssignedTeamId
- AiPredictedCategory
- AiPredictedPriority
- AiConfidenceScore
- RequiresAgentReview
- CreatedByUserId
- CreatedAt, UpdatedAt, ResolvedAt
- SlaDueAt

Category

- Id, Name, DefaultTeamId

Team

- Id, Name, Description, CreatedAt

TicketComment

- Id, TicketId, AuthorId, Message, IsInternal, CreatedAt

TicketAttachment

- Id, TicketId, FileName, FilePath, FileSize, ContentType, UploadedAt

TicketHistoryEntry

- Id, TicketId, FieldName, OldValue, NewValue, ChangedByUserId, ChangedAt

Authorization Rules

- POST /api/auth/register and POST /api/auth/login are public.
- All other controllers require JWT authentication.
- Customer can create tickets and view their own tickets.
- Agent can view assigned tickets, update status/priority/category, add comments, and request AI replies.
- Admin can manage tickets, teams, categories, and all users.
- Ticket access is enforced by Auth.RequireTicketAccess.

Data Flow Diagrams

Login flow

```
Customer UI -> AuthService.login() -> POST /api/auth/login
backend AuthController.Login:
  validate credentials
  issue JWT
  return { token, user }
Frontend:
  store token in localStorage
  store user profile
  update authenticated state
```

Create ticket flow

```
Customer UI -> TicketService.create() -> POST /api/tickets
backend TicketsController.Create:
  Auth.RequireRole(Customer)
  validate request
  create Ticket in Store.Tickets
  TicketHistoryService.Add(...)
  TicketJobDispatcher.EnqueueClassify(ticketId)
  return 201 Created
```

Ticket detail flow

```
Agent UI -> TicketService.get(id) -> GET /api/tickets/{id}
backend TicketsController.Detail:
  Auth.RequireTicketAccess(user, ticketId)
  build TicketDetailResponse with comments, attachments, history
  return 200 OK
```

AI reply flow

```
Agent UI -> TicketService.suggestReply(id) -> POST /api/tickets/{id}/ai-reply
backend TicketsController.SuggestReply:
  Auth.RequireAnyRole(Agent, Admin)
  validate ticket access
  AiReplyService.GenerateReplyAsync(ticket, customer)
  return { reply }
```

Assignment flow

```
Admin UI -> TicketService.assign(id, agentId, teamId) -> PUT /api/tickets/{id}/assign
backend TicketsController.Assign:
  Auth.RequireRole(Admin)
  validate agent and team
  update ticket.AssignedAgentId and AssignedTeamId
  set status = Assigned
  add history entry
  return 200 OK
```

Deployment Notes

- Backend host: http://localhost:5000
- Frontend API base URL: http://localhost:5000/api
- CORS policy allows Angular dev server origins on port 4200
- Attachments are stored on disk under AppFeatures.UploadPath
- Swagger is available in development environments